K-Nearest Neighbors (KNN) Definition

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression.

It works by finding the 'K' closest data points (neighbors) to a given input and making predictions based on their majority class (for classification) or average value (for regression).

K is a hyperparameter that determines how many neighbors are considered.

Key points:

- It is a non-parametric and instance-based learning algorithm.
- It uses distance metrics like Euclidean or Manhattan distance.
- Smaller K can lead to overfitting, while larger K can lead to underfitting.

In [1]:
```
# importing major libraries

import pandas as pd
import numpy as np
```

In [2]:
```
df = pd.read_csv('Cancer_Data.csv').drop(['id','Unnamed: 32'],axis=1)
```

In [3]:
```
df.head()
```

Out[3]:

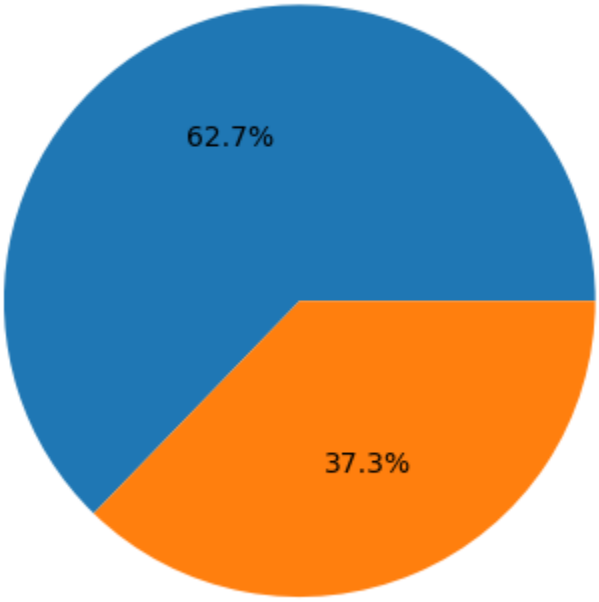
	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothi
0	M	17.99	10.38	122.80	1001.0	
1	M	20.57	17.77	132.90	1326.0	
2	M	19.69	21.25	130.00	1203.0	
3	M	11.42	20.38	77.58	386.1	
4	M	20.29	14.34	135.10	1297.0	

5 rows × 31 columns

In [4]:
```
df.columns

df.diagnosis.value_counts().index

df.diagnosis = df.diagnosis.map({'B':0, 'M':1})
```

In [5]:
```
x = df.iloc[:,1:]
y = df.diagnosis
```

In [6]:
```
import matplotlib.pyplot as plt
import seaborn as sns
plt.pie(df.diagnosis.value_counts(),autopct='%1.1f%%')
plt.show()
```



In [7]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,random_state=1,test_size=
knn = KNeighborsClassifier(n_neighbors=5)
knn
```

Out[7]:
```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

In [8]:
```
knn.fit(x_train,y_train)
```

Out[8]:
```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

In [9]:
```
from sklearn.metrics import accuracy_score,recall_score
y_pred = knn.predict(x_test)
print('accuracy ',accuracy_score(y_test,y_pred)*100)
print('recall ',recall_score(y_test,y_pred)*100)
```

```
accuracy  93.85964912280701
recall  88.09523809523809
```

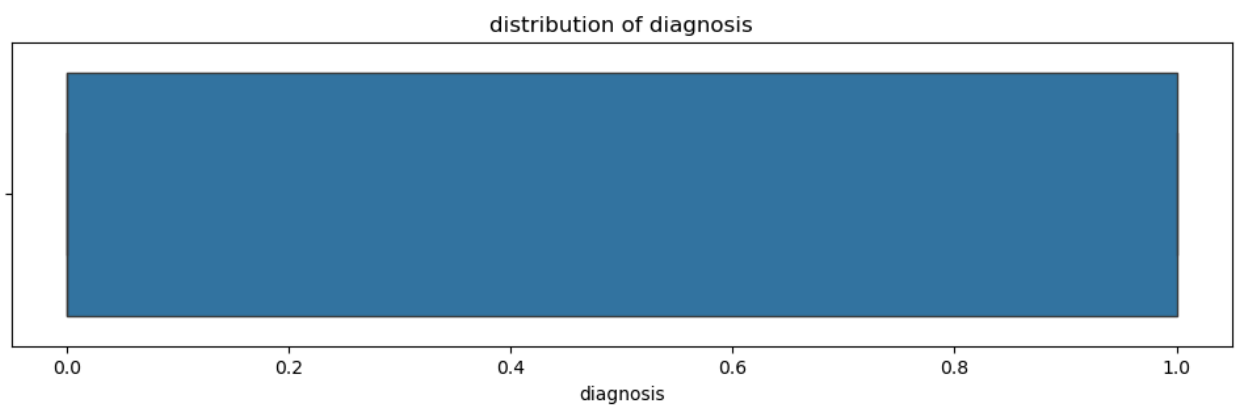
Drawback

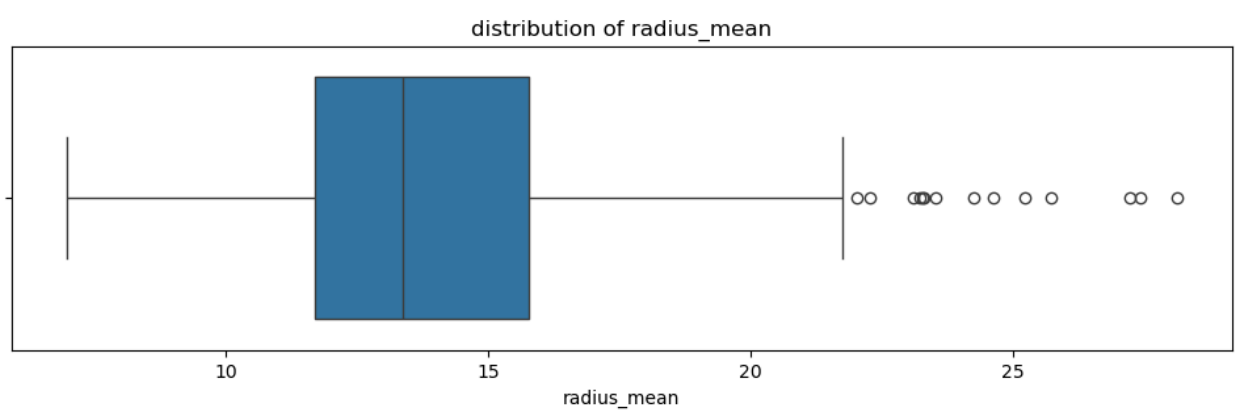
1. Slow on Large datasets
2. Sensitive towards outliers
3. Curse of Dimensionality
4. Feature Scaling --> standard scaler
5. Choosing k Value is difficult

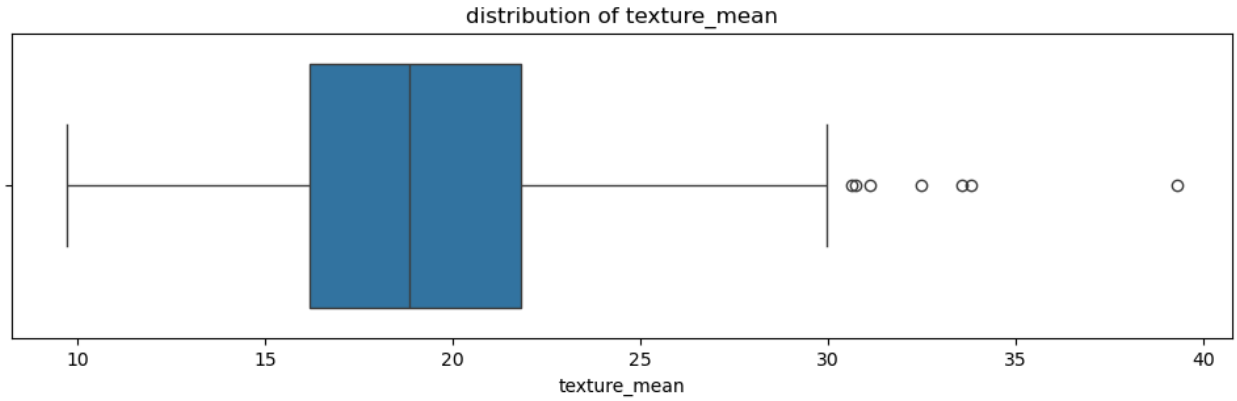
- K(BIG) --> UNDERFIT
- K(SMALL) --> OVERFIT

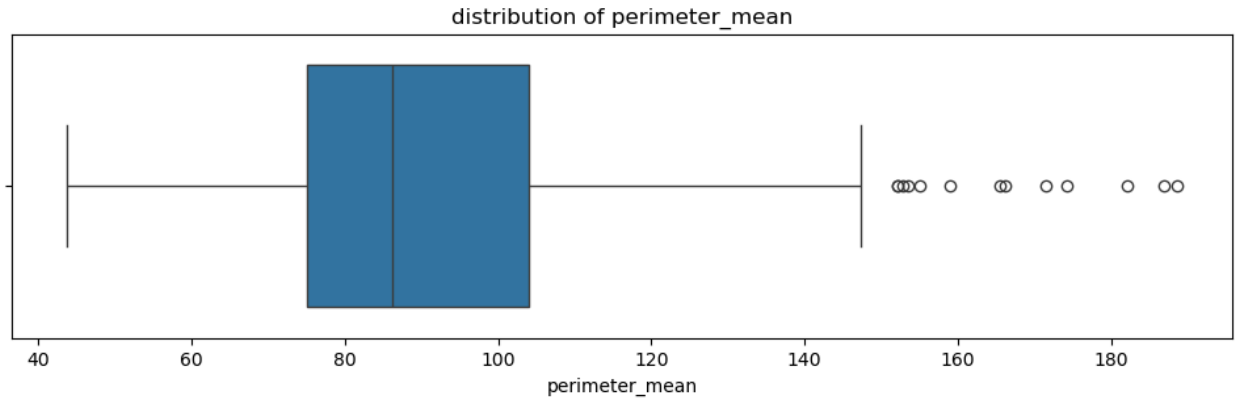
6. EXPENSIVE

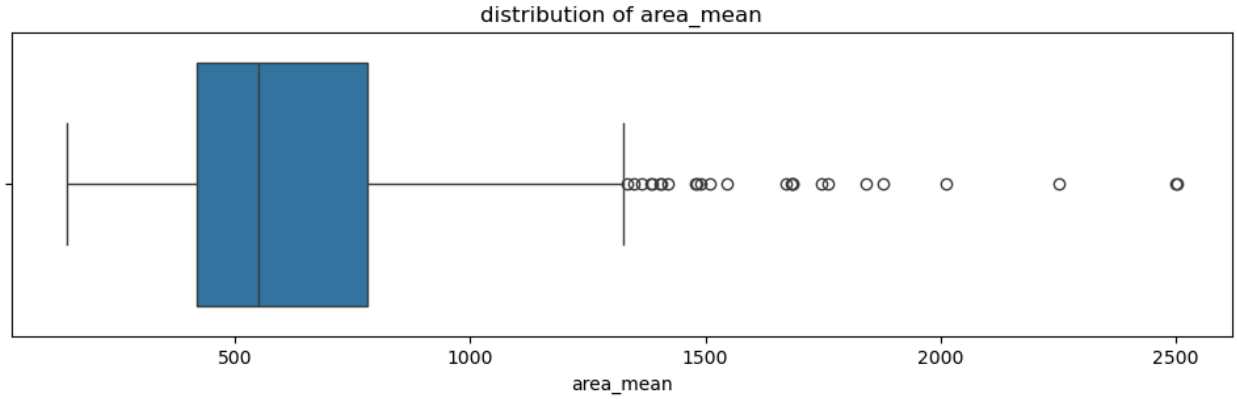
In [10]:
```
df
import seaborn as sns
import matplotlib.pyplot as plt

for i in df.columns:
    #print(i)
    plt.figure(figsize=(12,3))
    sns.boxplot(x=df[i])
    plt.title(f'distribution of {i}')
    plt.show()
```

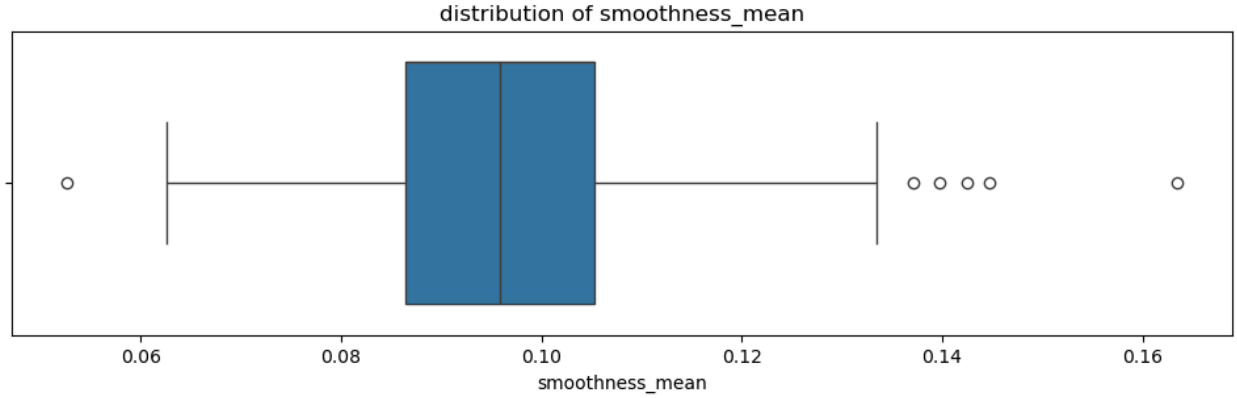


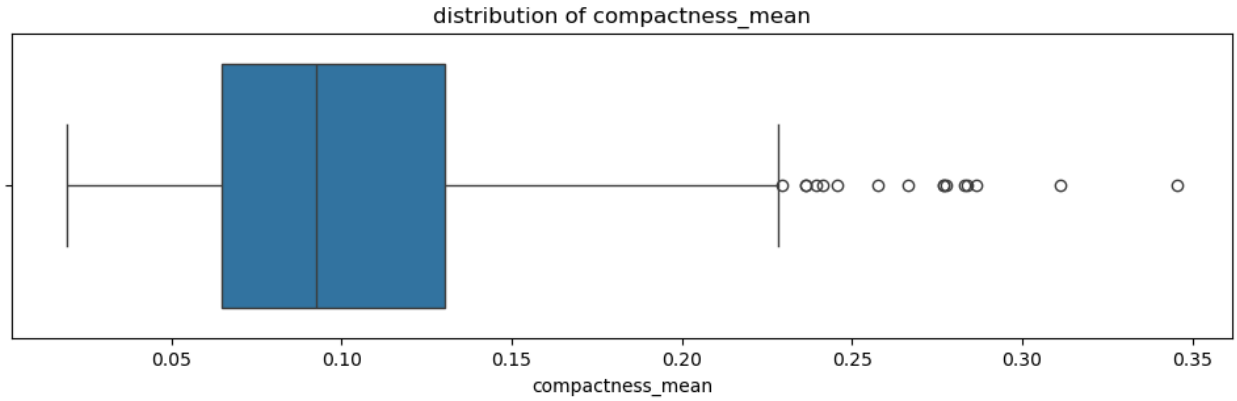


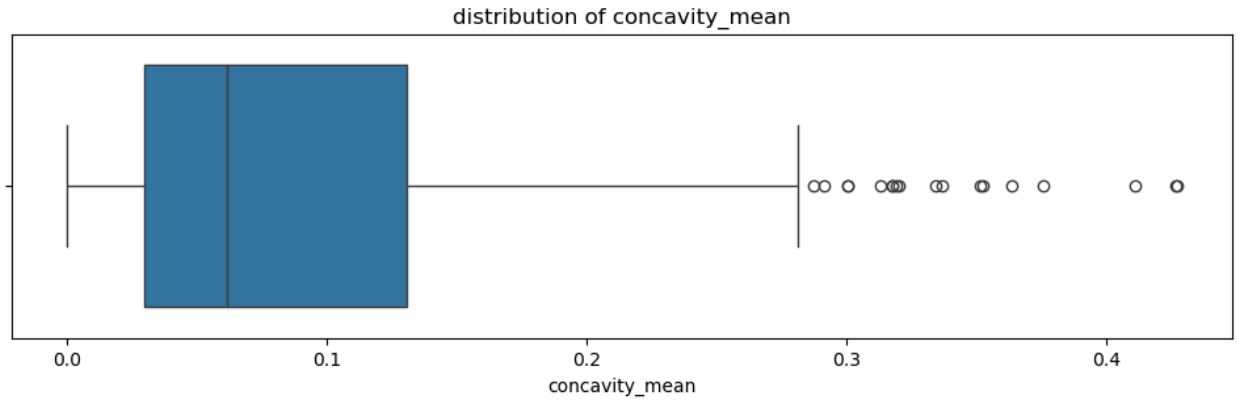

distribution of texture_mean
10
15
20
25
30
35
40
texture_mean

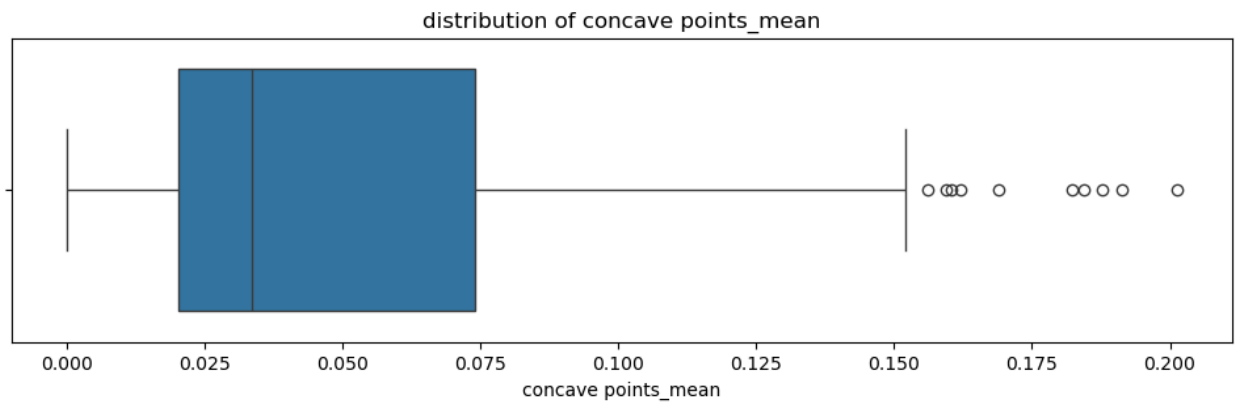

distribution of perimeter_mean
40
60
80
100
120
140
160
180
perimeter_mean

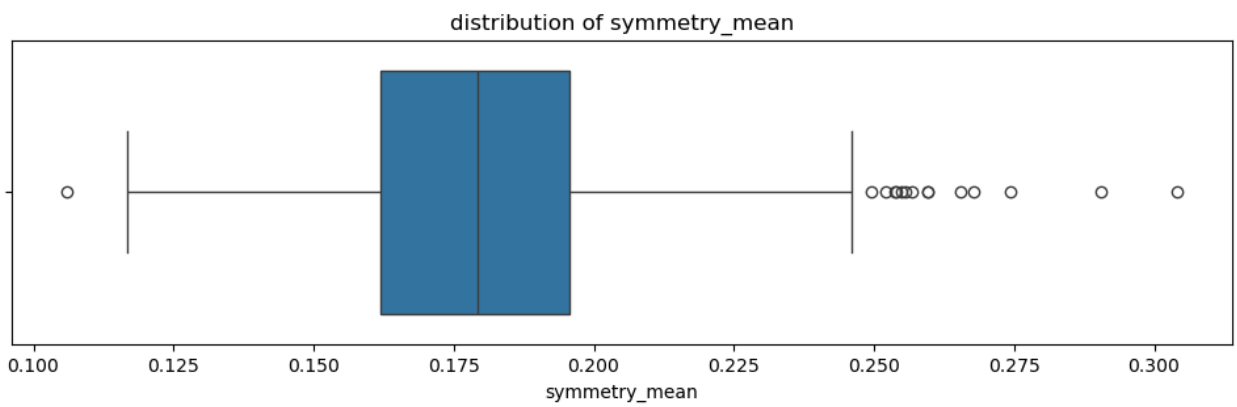

distribution of area_mean
500
1000
1500
2000
2500
area_mean

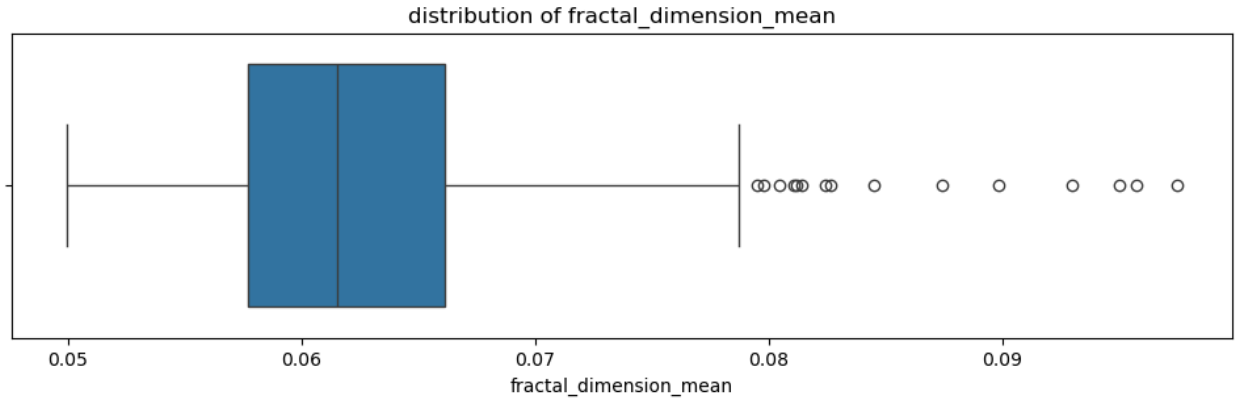

distribution of smoothness_mean
0.06
0.08
0.10
0.12
0.14
0.16
smoothness_mean

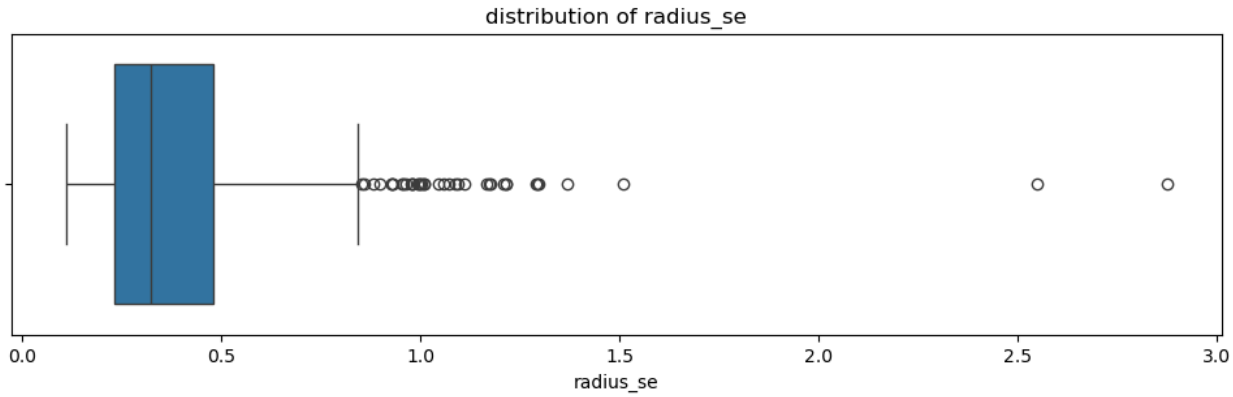

distribution of compactness_mean
0.05
0.10
0.15
0.20
0.25
0.30
0.35
compactness_mean

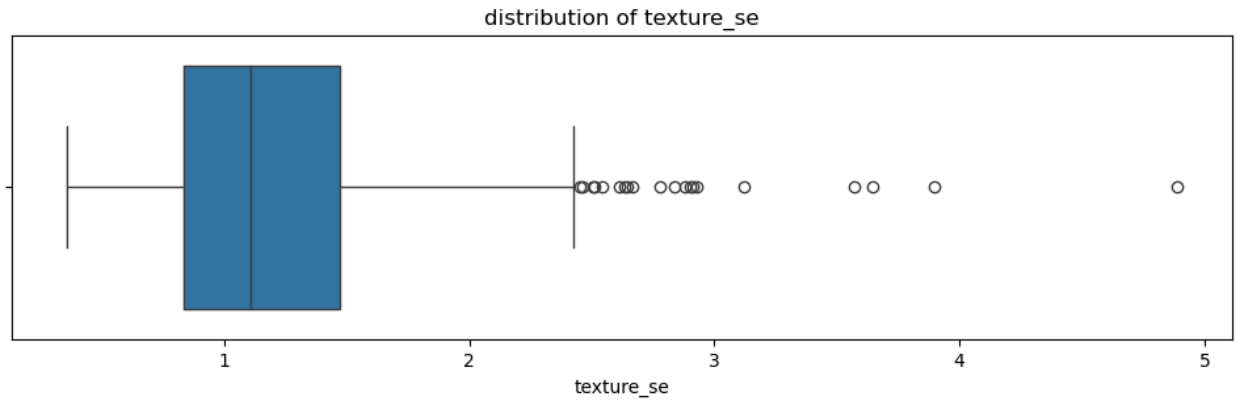

distribution of concavity_mean
0.0
0.1
0.2
0.3
0.4
concavity_mean

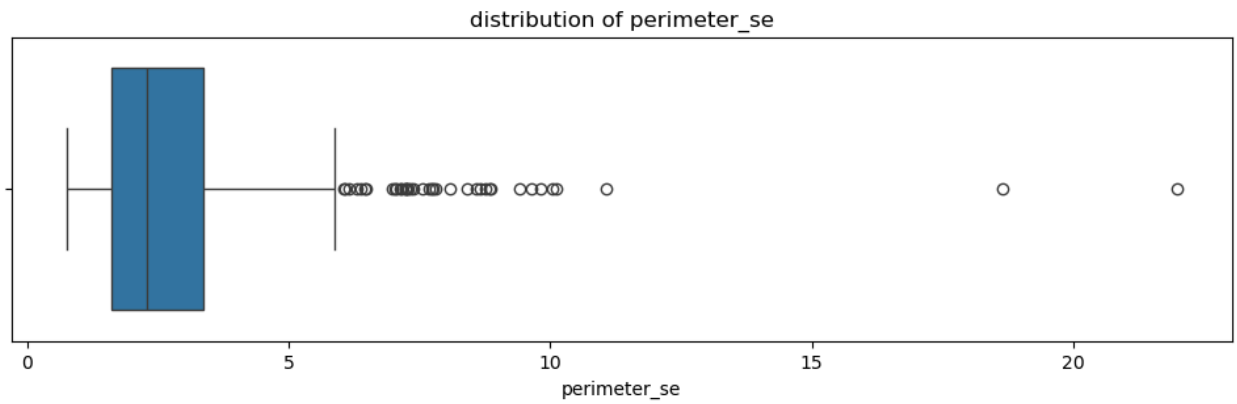

distribution of concave points_mean
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
concave points_mean

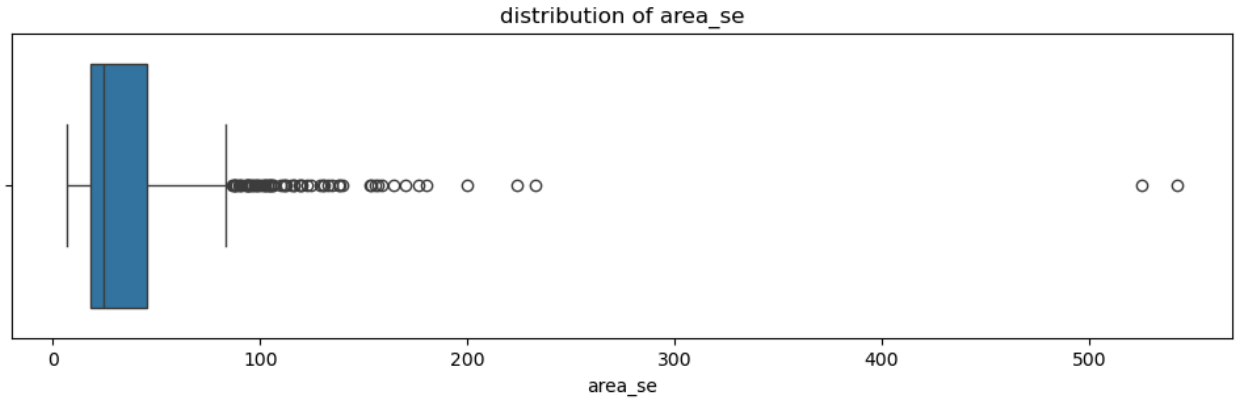

distribution of symmetry_mean
0.100
0.125
0.150
0.175
0.200
0.225
0.250
0.275
0.300
symmetry_mean

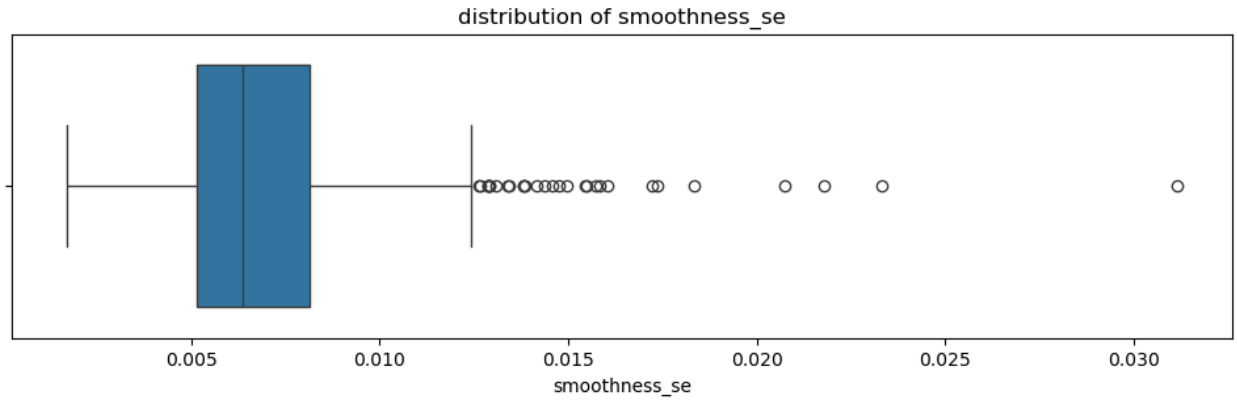

distribution of fractal_dimension_mean
0.05
0.06
0.07
0.08
0.09
fractal_dimension_mean

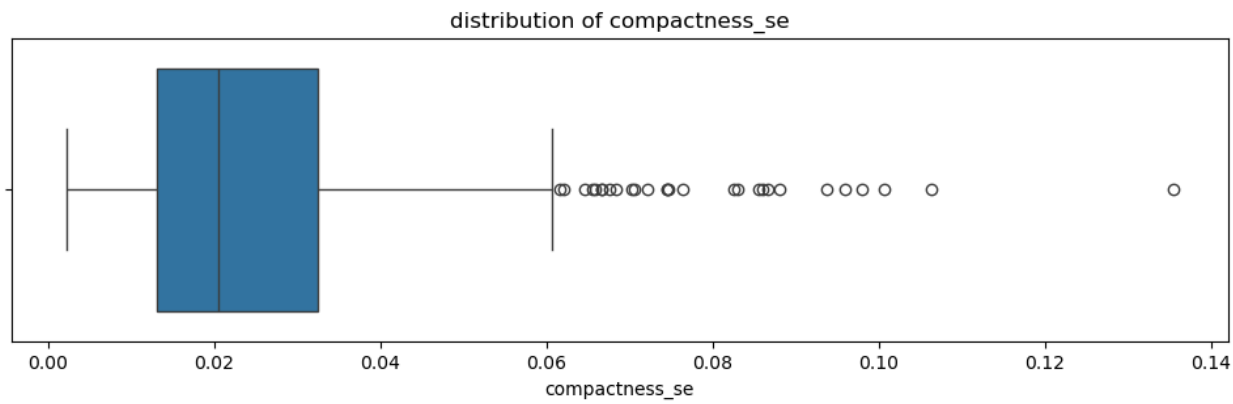

distribution of radius_se
0.0
0.5
1.0
1.5
2.0
2.5
3.0
radius_se

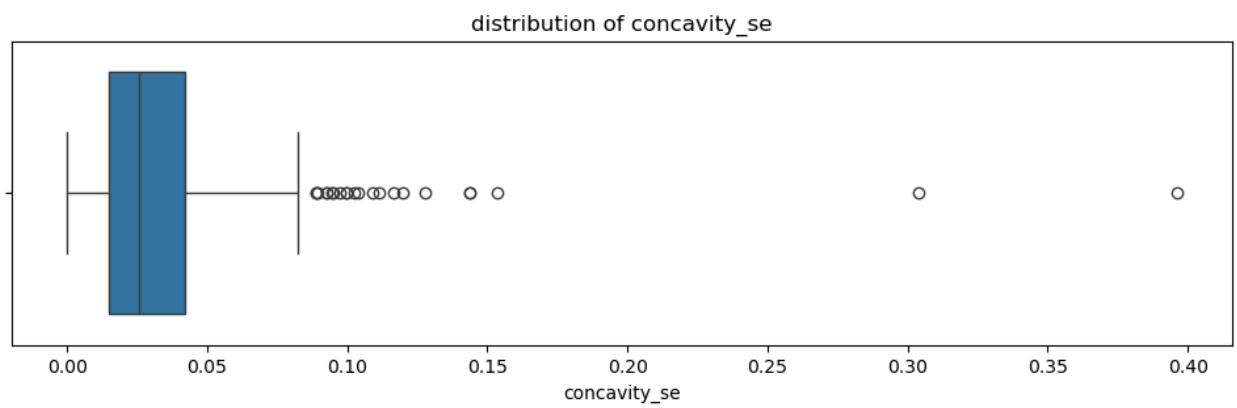

distribution of texture_se
1
2
3
4
5
texture_se

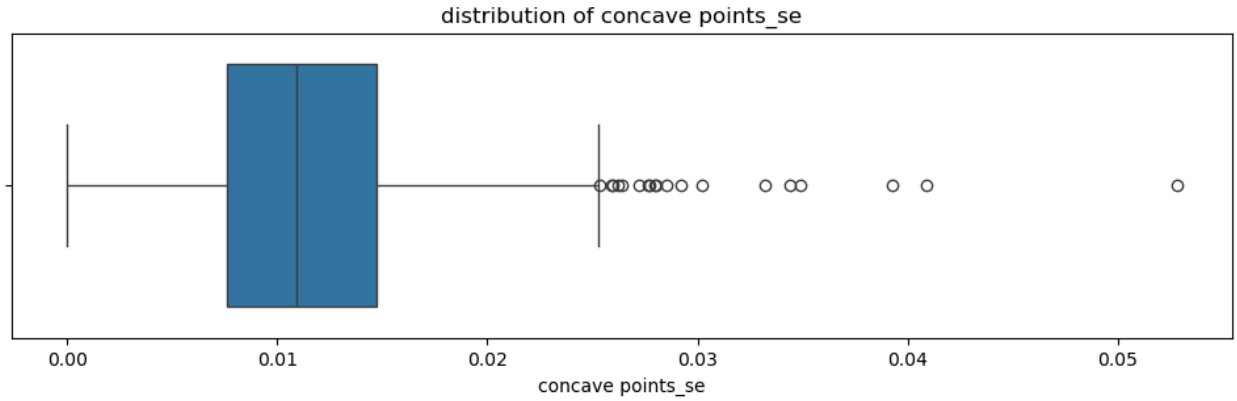

distribution of perimeter_se
0
5
10
15
20
perimeter_se

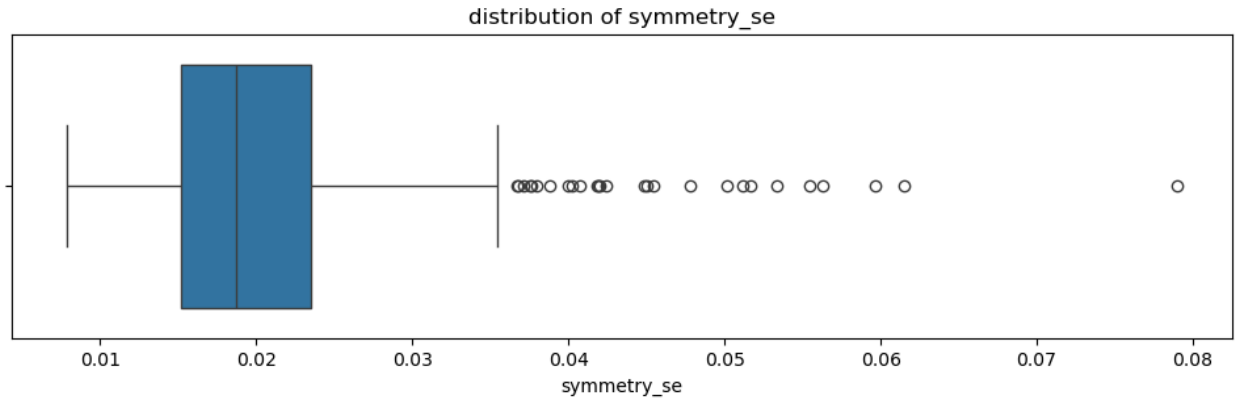

distribution of area_se
0
100
200
300
400
500
area_se

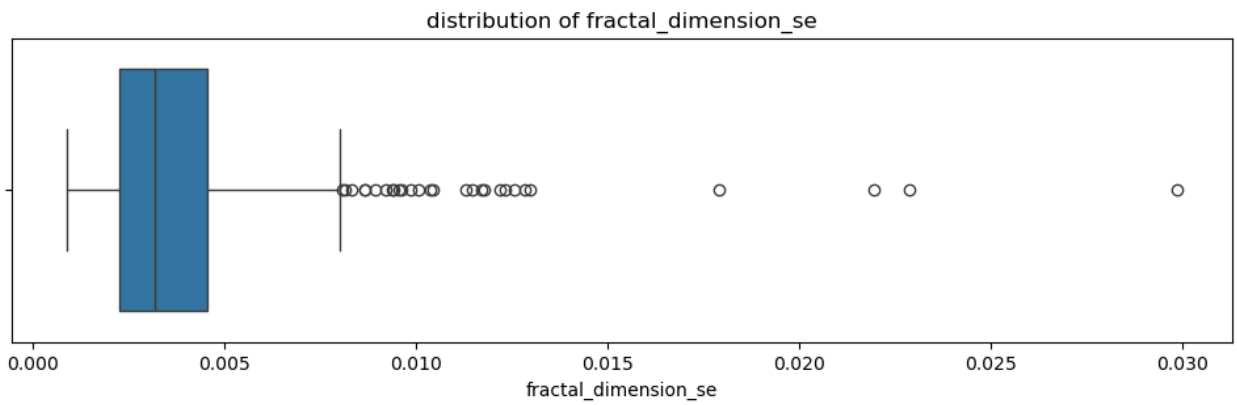

distribution of smoothness_se
0.005
0.010
0.015
0.020
0.025
0.030
smoothness_se

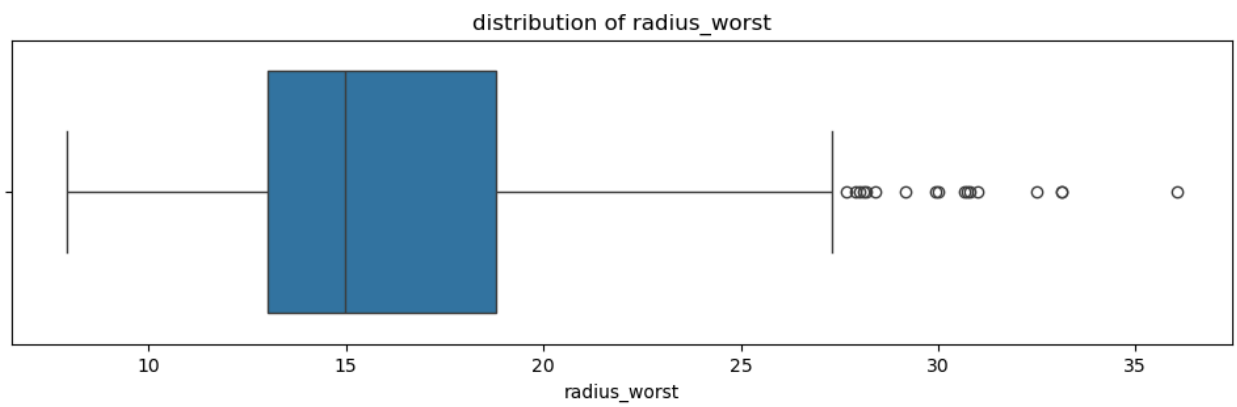

distribution of compactness_se
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
compactness_se

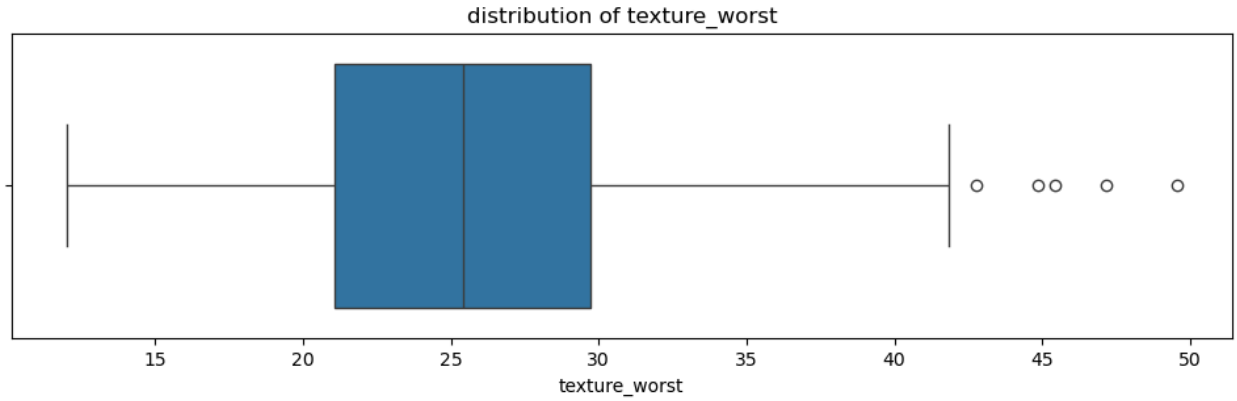

distribution of concavity_se
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
concavity_se

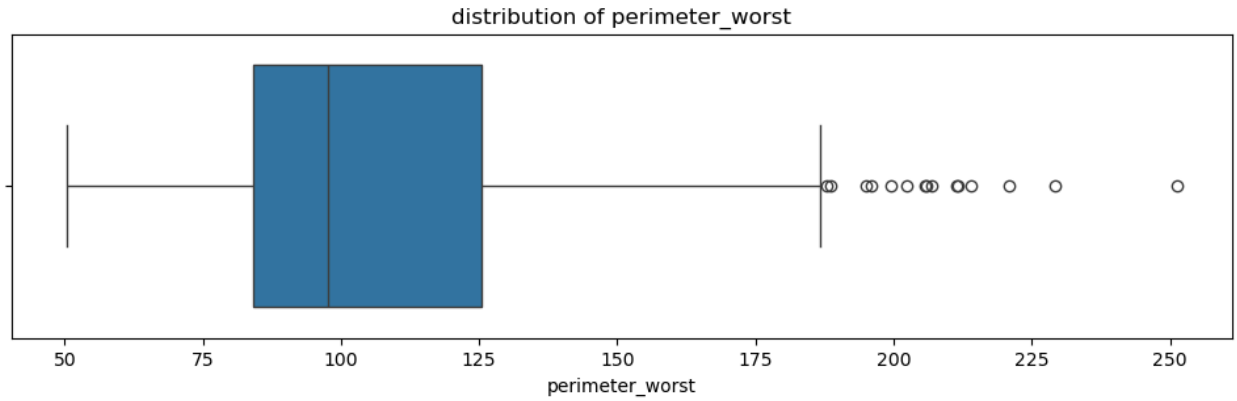

distribution of concave points_se
0.00
0.01
0.02
0.03
0.04
0.05
concave points_se

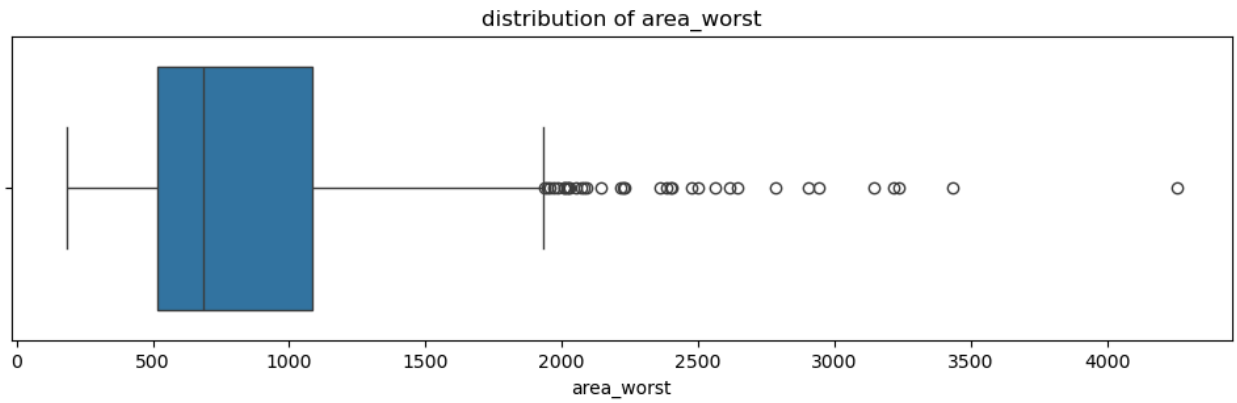

distribution of symmetry_se
0.01
0.02
0.03
0.04
0.05
0.06
0.07
0.08
symmetry_se

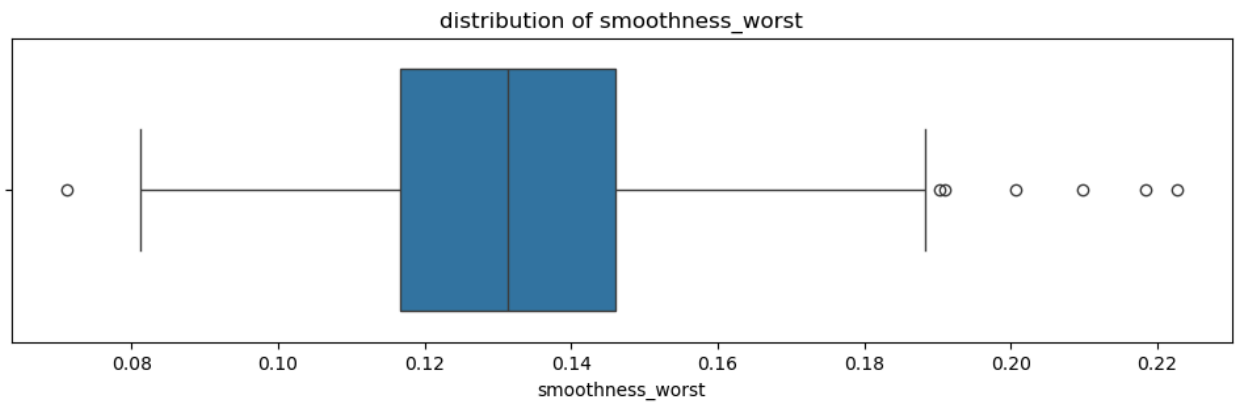

distribution of fractal_dimension_se
0.000
0.005
0.010
0.015
0.020
0.025
0.030
fractal_dimension_se

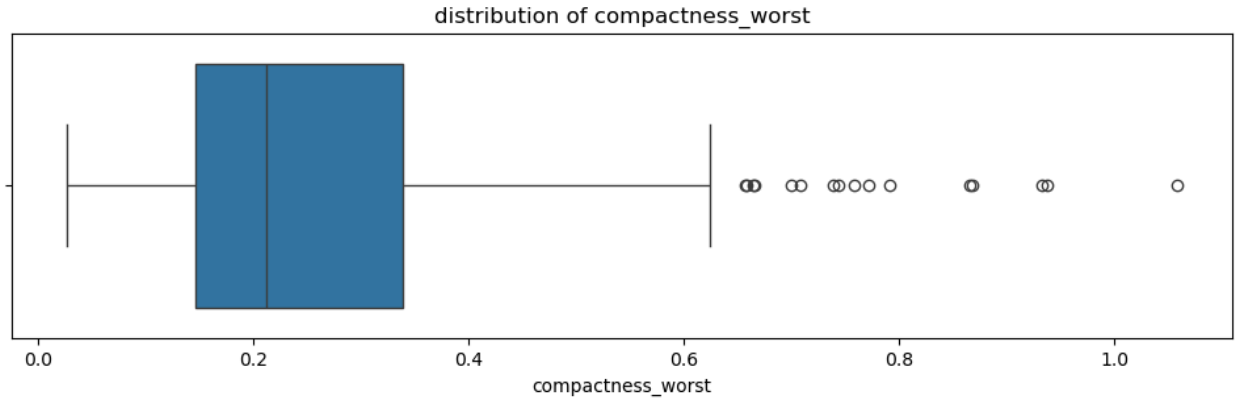

distribution of radius_worst
10
15
20
25
30
35
radius_worst

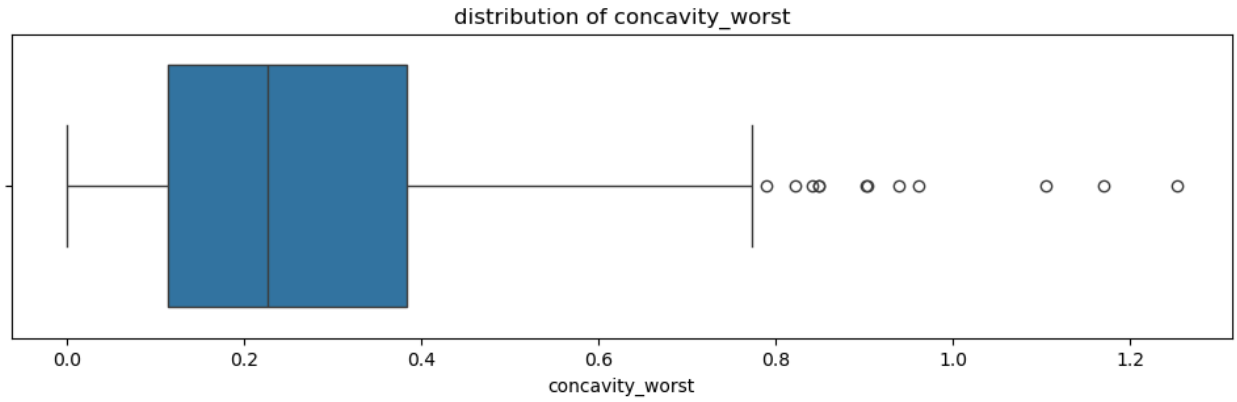

distribution of texture_worst
15
20
25
30
35
40
45
50
texture_worst

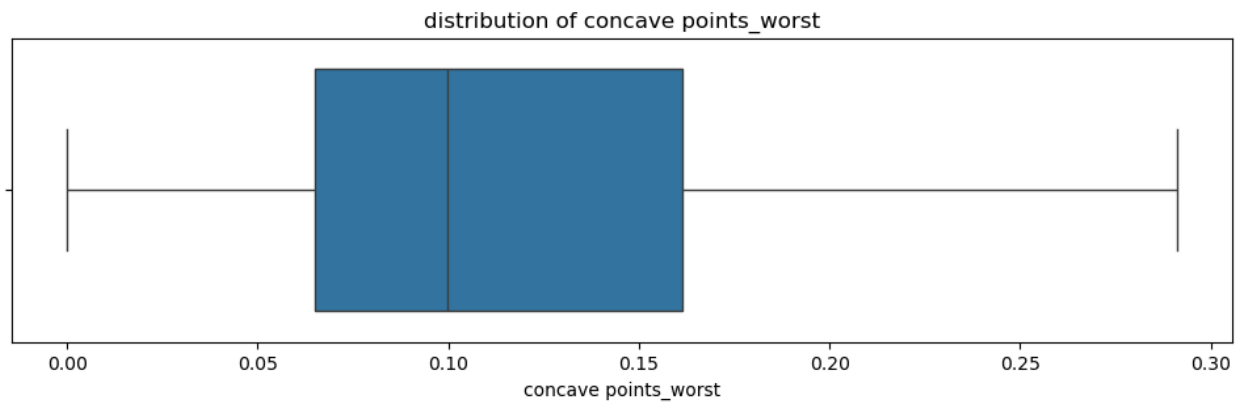

distribution of perimeter_worst
50
75
100
125
150
175
200
225
250
perimeter_worst

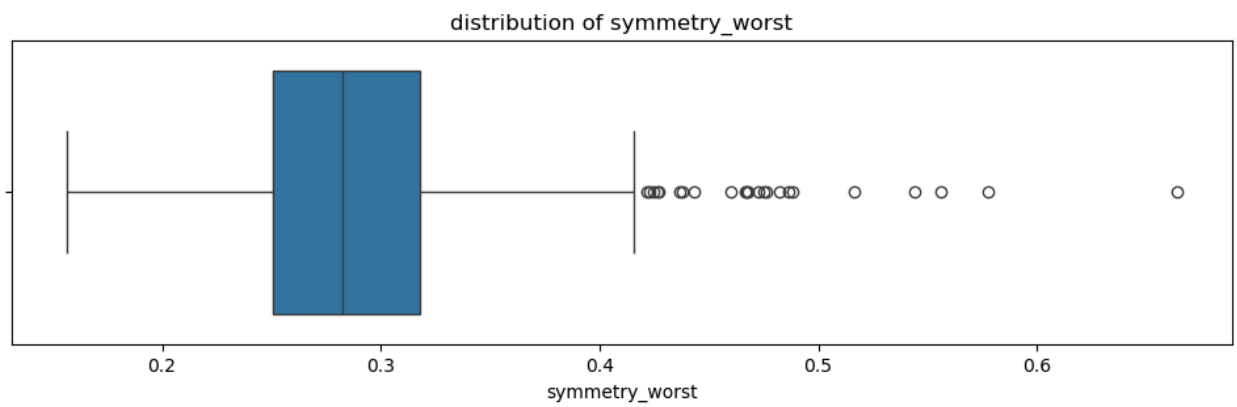

distribution of area_worst
0
500
1000
1500
2000
2500
3000
3500
4000
area_worst

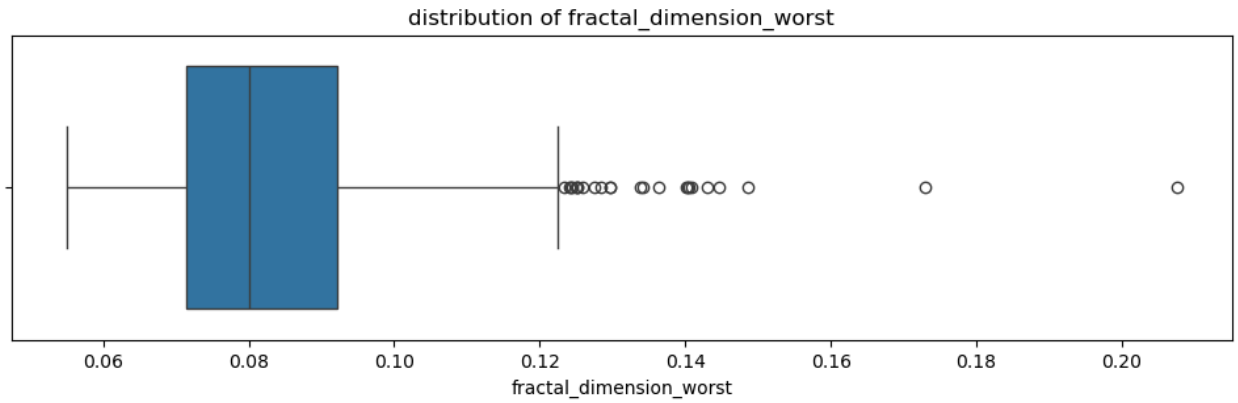

distribution of smoothness_worst
0.08
0.10
0.12
0.14
0.16
0.18
0.20
0.22
smoothness_worst


distribution of compactness_worst
0.0
0.2
0.4
0.6
0.8
1.0
compactness_worst


distribution of concavity_worst
0.0
0.2
0.4
0.6
0.8
1.0
1.2
concavity_worst


distribution of concave points_worst
0.00
0.05
0.10
0.15
0.20
0.25
0.30
concave points_worst


distribution of symmetry_worst
0.2
0.3
0.4
0.5
0.6
symmetry_worst



In [11]:
```
# 1,10,20,30,1000

from sklearn.preprocessing import RobustScaler

rs = RobustScaler()
x_train_scaled = rs.fit_transform(x_train)
x_test_scaled = rs.transform(x_test)
```

In [12]:
```
x_train_scaled = pd.DataFrame(x_train_scaled,columns=rs.get_feature_names_out(
x_test_scaled= pd.DataFrame(x_test_scaled,columns=rs.get_feature_names_out())
```

In [13]:
```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=15,weights='distance',algorithm='brute'
knn.fit(x_train_scaled,y_train)
knn.score(x_test_scaled,y_test)
```

Out[13]:
```
0.9649122807017544
```

In [14]:
```
knn
```

Out[14]:
```
KNeighborsClassifier
KNeighborsClassifier(algorithm='brute', n_jobs=-1, n_neighbors=15,
                     weights='distance')
```

In [15]:
```
x_train_scaled.describe().round()
```

Out[15]:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_m
count	455.0	455.0	455.0	455.0	4
mean	0.0	0.0	0.0	0.0	
std	1.0	1.0	1.0	1.0	
min	-1.0	-2.0	-1.0	-1.0	
25%	-0.0	-0.0	-0.0	-0.0	
50%	0.0	0.0	0.0	0.0	
75%	1.0	1.0	1.0	1.0	
max	3.0	4.0	3.0	5.0	

8 rows × 30 columns

In [16]:

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([
    ('scale',RobustScaler()),
    ('estimator',KNeighborsClassifier(n_neighbors=15,weights='distance',algori
])
pipe
```

Out[16]:

```
▸ Pipeline
  ▸ RobustScaler
  ▸ KNeighborsClassifier
```

In [17]:

```
pipe.fit(x_train,y_train)
from sklearn.metrics import accuracy_score,recall_score
print(accuracy_score(y_test,pipe.predict(x_test))*100)
print(recall_score(y_test,pipe.predict(x_test))*100)
```

```
96.49122807017544
90.47619047619048
```

In [18]:

```
pip install joblib
```

```
Requirement already satisfied: joblib in c:\users\vipul\anaconda3\lib\site-pack
ages (1.4.2)
Note: you may need to restart the kernel to use updated packages.
```

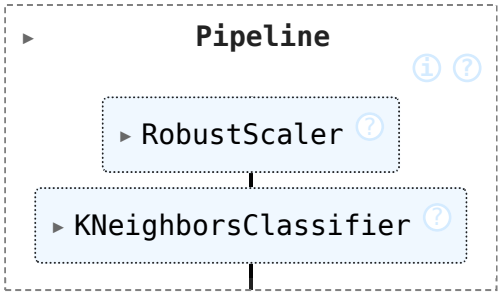
In [19]:

```
import joblib
joblib.dump(pipe,'knn_pipe.pkl')
```

Out[19]: ['knn_pipe.pkl']

In [20]:
```
joblib.load('knn_pipe.pkl')
```

Out[20]:



In [ ]: